

Java vs Kotlin — Manejo de Errores y Excepciones (Versión Extendida)

Filosofía general

Java: combina excepciones checked (IOException, SQLException) y unchecked (IllegalArgumentException, Kotlin: todas las excepciones son unchecked; fomenta Result, sealed y manejo funcional.

try/catch/finally

```
Java:
try {
    doWork();
} catch(IOException e) {
    log.error("IO", e);
} finally {
    cleanup();
}

Kotlin:
try {
    doWork()
} catch(e: IOException) {
    println("IO: $e")
} finally {
    cleanup()
}
```

Multi-catch

```
Java (multi-catch):
try { ... }
catch(IOException | SQLException e) {
    handle(e);
}

Kotlin: no multi-catch, usar varios bloques catch:
try { ... }
catch(e: IOException) { ... }
catch(e: SQLException) { ... }
```

Recursos

```
Java try-with-resources:
try (BufferedReader r = Files.newBufferedReader(path)) {
    return r.readLine();
}

Kotlin use:
Files.newBufferedReader(path).use { r ->
    return r.readLine()
}
```

Manejo funcional (Kotlin)

```
val result = runCatching { risky() }
result.onSuccess { println(it) }
      .onFailure { e -> println("Error: $e") }

val value = result.getOrElse { "fallback" }
```

Sealed classes

```
Kotlin sealed interface:
sealed interface FileError {
    data class NotFound(val path: String): FileError
    data class Permission(val user: String): FileError
}

fun handle(error: FileError) = when(error) {
    is FileError.NotFound -> println("No encontrado: ${error.path}")
    is FileError.Permission -> println("Sin permiso: ${error.user}")
}
```

Buenas prácticas

Java:

- Usar checked para E/S, unchecked para errores de lógica.
- Preferir try-with-resources.
- Evitar catch(Exception) genérico.

Kotlin:

- Preferir Result o sealed para errores de dominio.
- use { } en recursos.
- Manejar interoperabilidad con @Throws si Java lo necesita.